PlacementRadar

Job Intelligence & Outreach Automation for College Placement Cells

Developer: Shubham Chavan **Institution:** K. J. Somaiya Institute of Management **Live Application:** https://placementradar.onrender.com **Repository:** https://github.com/shubechavan/placementradar

---

1. Introduction

PlacementRadar is a full-stack web application that automates the entire campus placement outreach pipeline — from discovering job openings across dozens of portals, to finding the HR contact behind each company, to sending and tracking outreach emails — from a single dashboard.

Campus placement work is fundamentally a **sales funnel run by students**. A placement committee must find companies that are hiring, identify a human being at each company who can be contacted, reach out professionally, follow up, and keep a shared record of who spoke to whom. Every one of these steps is currently done by hand, in spreadsheets, by volunteers who also have coursework.

PlacementRadar treats that funnel as an engineering problem. It runs the discovery and enrichment stages automatically, scores every contact it finds by how likely it is to be correct and reachable, and gives the committee a shared CRM so the human effort is spent on conversations rather than on data entry.

The system runs at **zero monthly infrastructure cost**, using free tiers

of Render, MongoDB Atlas, and GitHub Actions — a deliberate constraint, because a tool a student committee cannot afford to keep running is a tool that dies at the end of the semester.

---

2. Problem Statement

A placement committee's workflow breaks down at four distinct points.

Job discovery is fragmented. Openings relevant to one institute are scattered across Naukri, Internshala, LinkedIn, Indeed, Foundit, Shine, Apna, and a long tail of startup and remote boards. No single portal covers the field, so a committee member either checks fifteen sites by hand or accepts an incomplete picture. In practice they accept the incomplete picture.

Finding the HR contact is the real bottleneck. A job listing rarely includes a recruiter's email or phone. That information exists — on the company's own careers page, in business directories, in corporate filings — but locating it means manually searching per company. At roughly five minutes per company, a list of three hundred companies is twenty-five hours of work before a single email is sent.

Contact data decays and cannot be trusted. Addresses bounce, numbers disconnect, listings go stale. Worse, contacts are frequently *mis-attributed*: an address genuinely belonging to one employer gets filed under another with a similar name. Measurement on this project's own corpus found that **23.5% of contacts attached to an identified company sat on a company whose domain they did not match** — an error invisible to any count or summary, because the totals look correct either way.

There is no shared institutional memory. Without a common record, two members email the same company in the same week, follow-ups are missed, and when the committee changes hands each year the relationship history leaves with it. The placement head has no visibility into who is actually doing the work.

The underlying problem is not that any single step is hard. It is that the whole pipeline is **manual, unverified, and unrecorded** — so effort does not compound, and quality cannot be measured.

---

3. Proposed Solution

PlacementRadar restructures the pipeline into four automated stages with a human in the loop at the points where judgement is genuinely required.

Stage 1 — Scrape

A unified scraping engine pulls openings from **20+ job boards** in parallel, plus any custom sites the committee defines. Results are normalised into a single schema and deduplicated by job URL, so the same posting appearing on four portals becomes one row.

Stage 2 — Enrich

For each company, multiple sources run concurrently to recover contact details: the company's own website and a deep crawl of its careers and contact pages, sitemap-driven page discovery, business directories, and corporate registries. Phone numbers are validated with Google's **libphonenumber**; email addresses against an RFC validator and a live **DNS MX lookup** to confirm the receiving domain actually accepts mail.

Stage 3 — Score and verify

Every contact carries a **confidence score** derived from independent evidence: whether it sits on the company's own confirmed domain, whether a named person is attached, whether the role is HR-related, whether MX verification passed, and whether the same value appears across several unrelated companies (the signature of a shared switchboard rather than a direct line). Contacts that fail attribution

checks are **quarantined, not deleted**, so a false negative can be recovered.

Stage 4 — Act and track

The committee sends templated outreach over Gmail, auto-queues non-responders for follow-up, and logs every call, email, and WhatsApp message against the company in a shared CRM. Leads move across a Kanban pipeline, and the placement head sees a live leaderboard of committee activity.

The design principle that shapes all four

Anything that produces an identity or attribution decision gets a review queue, not a heuristic. Deciding whether `contact@simulab.in` belongs to `simulab.com` is not a problem string-matching can solve — `advice@rethink.org` and `rethink.com` are *different organisations* with identical surface similarity. Where the data genuinely cannot settle the question, the system surfaces the decision to a human with the evidence attached, rather than guessing and recording the guess as fact. This is why the tool's output can be trusted enough to email a stranger from.

---

4. The Social Cause

What it is

Equalising access to placement opportunities across institutions.

Campus recruitment is structurally unequal, and the inequality is not primarily about student ability. Elite institutions have dedicated, salaried placement offices, decades of accumulated corporate relationships, and recruiters who arrive unprompted. Everyone else — tier-2 and tier-3 colleges, newer institutes, regional campuses — runs placements on a

volunteer student committee with a spreadsheet and no inherited contact list.

The gap this creates is one of **reach, not merit**. A capable student at an under-resourced institute is less likely to be placed not because they are less prepared, but because no one from their college ever contacted the company that would have hired them. The bottleneck is administrative capacity, and administrative capacity is exactly what software can supply.

How PlacementRadar addresses it

- **It replaces inherited relationships with discoverable ones.** A committee with no alumni network can still find and reach hiring companies, because the tool derives its contact list from public sources rather than from institutional history.
- **It makes the work survive the committee.** Placement committees turn over annually, and the relationship history usually leaves with the outgoing batch. A shared CRM turns individual effort into an institutional asset that the next cohort inherits.
- **It costs nothing to run.** The $0/month architecture is a social design decision, not only an engineering one. A tool that requires a budget line excludes precisely the institutions that most need it. Any college can deploy this on free tiers.
- **It returns student time to studying.** The twenty-five hours a committee spends looking up contacts is time taken from the coursework those students are at college to complete. Automating the mechanical portion is a direct transfer of time back to education.

The measurable outcome is simple: **more companies contacted per committee-hour, by institutions that cannot buy their way to the same reach.**

---

5. AI Implementation

An honest account of what is intelligent in this system, what is deterministic, and where machine learning genuinely applies.

What is implemented today: deterministic decision systems

The intelligence currently in PlacementRadar is **algorithmic and rule-based, not a trained model** — and for the identity and attribution decisions at its core, that is a deliberate choice rather than a limitation. These decisions must be *auditable*: when the system claims an email belongs to a company, a committee member needs to see the evidence before emailing a stranger. A deterministic rule can be inspected, tested, and corrected; a model's confidence score cannot be argued with.

Weighted confidence scoring. A pure scoring function combines independent signals — own-domain match, MX verification, named person, HR-family role, cross-company spread — into a single 0–1 score. It is implemented as one isolated function so weights can be tuned without touching any scraper.

Entity resolution with a corroboration gate. Matching a company name to its real web domain is a classic entity-resolution problem, and name similarity alone fails badly — three different firms in this corpus share the name "Genesis". The system resolves a candidate domain only when the page independently corroborates the company through its `<title>`, `og:site_name`, or footer copyright, and rejects **tautological evidence** (a page whose only "proof" is echoing back the domain that was guessed from the name).

Content-relevance verification. Scrapers verify that a page actually answers the query that was asked, rather than trusting an HTTP 200. This caught a real defect: a job board silently redirecting unknown search terms to a generic listing page and serving it successfully, so a search for "software engineer" returned sales and marketing roles.

Classification and validation. Role classification (technical vs non-

technical), role-account detection (`info@`, `hr@`), disposable-domain screening, and parked-page detection are all keyword and pattern systems, backed by libphonenumber and DNS verification for ground truth.

Where machine learning genuinely belongs — the honest roadmap

These are **not yet implemented**, and the project is explicit about that distinction.

Threshold derivation from labelled data. The current scoring weights are informed estimates, and analysis showed they are not yet calibrated — identity resolution dominates the ranking roughly sevenfold over contact quality. The correct fix is supervised: hand-label a stratified sample of contacts (`good` / `wrong-company` / `dead`), compute precision at each cutoff, and derive both the threshold and the weights from measured outcomes. **A labelling sheet has been prepared; this is the immediate next step.** The distinction matters — a threshold fitted to unvalidated weights relocates the problem rather than solving it.

LLM-assisted attribution review. The contact-to-company attribution problem is the strongest genuine case for a language model. Deciding whether `signify.com` legitimately serves `philips.com` requires world knowledge — Signify is the former Philips Lighting — that no string metric encodes. An LLM could pre-classify the review queue and surface a rationale, with a human retaining the final decision.

Semantic job-role matching. Keyword classification cannot tell that "SDE-1" and "Graduate Software Engineer" are the same role, or match a posting against a student's actual skill profile. Embedding-based similarity is the natural fit.

Outreach personalisation and reply classification. Generating a context-aware opening line per company, and classifying replies as interested / not-now / rejection to drive follow-up automatically.

> **A note on method.** The most valuable finding in this project was not produced by any algorithm: aggregate metrics looked correct while

> the underlying records were wrong. Mis-merged companies, contacts filed under the wrong employer, and a scraper returning an unrelated page all passed every summary statistic. They were found by opening individual records and reading them. Any future ML component must be evaluated the same way — on inspected examples, not only on aggregate accuracy.

---

6. Future Scope

Near term — closing what is already measured

- Derive the confidence threshold and weights from the labelled sample, replacing estimated values with measured ones.
- Resolve the outstanding contact-to-company attributions through the review queue.
- Complete human domain resolution for companies where automated discovery reached its ceiling — measurement showed roughly 60% of the unresolved population is unreachable by automation alone and requires human lookup.

Medium term — capability

- **Student-facing portal:** students see opportunities matched to their profile and apply through the system, closing the loop between committee outreach and placement.
- **Resume-to-role matching** using embeddings, replacing keyword filtering.
- **Analytics for the placement head:** conversion by sector, response rates by outreach template, historical comparison across placement seasons.
- **ATS integration.** Measurement showed only 1.6% of companies in this corpus expose a standard ATS API, so this is a long-tail enhancement rather than a primary channel — a finding worth recording, since the original plan assumed otherwise.

Long term — scale and reach

- **Multi-institution deployment** with isolated tenant data, so any college can run its own instance.
- **Shared verified-contact commons:** an opt-in pool where participating institutions contribute verification outcomes (this address bounced; this recruiter has moved), so data quality improves for everyone as more colleges use it. This is where the social cause compounds — the network effect favours smaller institutions most.
- **WhatsApp Business API** integration, given its dominance in Indian recruitment communication.
- **Alumni-network graph:** surfacing which companies employ the institute's alumni, the single strongest predictor of a successful placement conversation.

---

7. Technologies Used

Backend

Technology	Role
Python 3.10+	Core application language
Flask	Web framework and REST API
PyMongo	MongoDB driver
BeautifulSoup4 + lxml	HTML parsing for scrapers
Requests	HTTP client
concurrent.futures	Parallel source execution

Data & Validation

Technology	Role

MongoDB Atlas	Primary datastore and cross-process message bus
libphonenumber (`phonenumbers`)	Phone parsing, validation, type detection
email-validator	RFC-compliant address validation
dnspython	MX record verification
pandas	Corpus analysis and CSV processing

Frontend

Technology	Role
HTML5 / **CSS3** / **Vanilla JavaScript**	Single-page dashboard, no framework dependency
Chart.js	Analytics visualisation
Kanban pipeline UI	Drag-and-drop lead management

Security & Authentication

Technology	Role
Werkzeug (PBKDF2)	Password hashing
Flask sessions	Signed session cookies with hardened flags
Custom rate limiter	Sliding-window per-IP and per-email login throttle
Environment-only secrets	Credentials never persisted to the database

DevOps & Infrastructure

Technology	Role
Docker	Containerised, reproducible builds
GitHub Actions	CI/CD *and* as a free distributed scraping worker (2 CPU / 7 GB)
Render	Application hosting
Gmail SMTP	Outreach delivery

Engineering Practices

Practice	Purpose
Benchmark harness	Per-source yield measurement with an isolated HTTP cache
Database write guard	Asserts host, database, *and* collection before any write; dumps a restore point first
Append-only contact store	Re-scraping can only grow the contact set, never overwrite it
Rebuild-over-mutate	Corrections produce a fresh collection; prior versions are retained untouched
Per-source instrumentation	Distinguishes "blocked" from "found nothing" — sources with zero yield are reported, never hidden

Total monthly infrastructure cost: $0.

---

8. Conclusion

PlacementRadar began as an automation exercise — replacing a manual contact-hunting workflow with a scraper — and became something more useful: a demonstration that in data pipelines, **collection is the easy half and verification is the hard half.**

The system today discovers jobs across 20+ portals, maintains a company contact store with domain-first identity resolution, verifies email deliverability via MX records, scores every contact on independent evidence, and gives a placement committee a shared CRM with accountability — at zero infrastructure cost.

The most instructive outcome was methodological. Early in development, every aggregate metric indicated the system was working. Opening individual records showed otherwise: companies merged under a single identity, contacts filed against the wrong employer, a scraper confidently returning an unrelated page at HTTP 200. **None of these moved a summary statistic enough to notice.** They were found only by reading records one at a time, and fixing them required building verification into the pipeline rather than bolting it on — confirmation gates, relevance assertions, quarantine over deletion, and review queues wherever the data genuinely could not settle a question.

That principle is what makes the tool trustworthy enough to act on. A placement committee emailing a stranger on the system's recommendation is staking the institute's reputation on it. A contact list that is 80% right is not 80% as useful as a correct one — it is actively harmful, because there is no way to tell which 80%.

The remaining work is honest and specific: the confidence threshold must be derived from labelled data rather than estimated, outstanding attributions need human decisions, and the companies beyond automated reach need manual lookup. None of these are hidden behind a passing test.

What began as a tool to save a placement committee twenty-five hours of data entry is, more fundamentally, an argument that **access to opportunity should not depend on which institution happens to have inherited the right phone numbers** — and that this is a gap software can close.

Document prepared for academic submission. All figures cited are measured from the project's own corpus, not estimated.